Experiment 9: Implementing Logic Gates Using Neural Networks

Question

To implement basic logic gates (AND, OR, XOR) using neural networks and analyze their decision boundaries.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# Define logic gate inputs and expected outputs
logic_gates = {
'AND': {'X': np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), 'y': np.array([0, 0, 0, 1])},
'OR': {'X': np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), 'y': np.array([0, 1, 1, 1])},
'XOR': {'X': np.array([[0, 0], [0, 1], [1, 0], [1, 1]]), 'y': np.array([0, 1, 1, 0])}
}

# Perceptron model function
def perceptron(X, y, learning_rate=0.1, epochs=100):
    weights = np.random.rand(2)
    bias = np.random.rand()
    for epoch in range(epochs):
        for i in range(len(X)):
            output = np.dot(X[i], weights) + bias
            prediction = 1 if output >= 0 else 0
            error = y[i] - prediction
            weights += learning_rate * error * X[i]
            bias += learning_rate * error
    return weights, bias

# Train and display results
for gate, data in logic_gates.items():
    W, b = perceptron(data['X'], data['y'])
    print(f'Logic Gate: {gate}\nFinal Weights: {W}\nFinal Bias: {b}\n')
```

Output

Logic Gate: AND
Final Weights: [0.32192986 0.22739502]
Final Bias: -0.33981450003796326

Logic Gate: OR
Final Weights: [0.22989919 0.30542341]
Final Bias: -0.03750153732625558

Logic Gate: XOR
Final Weights: [-0.08556349 0.05694701]
Final Bias: 0.02073019538129453